

第050讲实验指南 数据划分与评价对象

1 这次实验要交出什么

你将从同一份60设备×48天的数据构造四份可解释的评价协议，亲手算完四行模型的一步更新，运行12个主例拟合和960个重复拟合，核验独立库结果，再解释十幅图。实验重点是目标、信息和独立单位，不是找最高分。

建议纸笔约45至60分钟，代码与图约60至90分钟，研究方案约30分钟。请先读讲义第1至7节；手算较生疏时，再读第8至9节。完整答案单独放在answers.pdf；不要用对照答案替代自己的计算记录。只阅读三个PDF无需安装软件。

交付作业应有五部分：一段准确评价目标；四份划分的计数与边界；逐样本前向、损失、导数和下一步前向；实际执行输出与独立参照；保留负面结果和局限的报告。解释为什么一个分数不能搬到另一个总体，是核心验收项。

2 文件地图

文件	用途
lecture.md 与 lecture.pdf	完整原理、十张图与E1至E16
lab.md 与 lab.pdf	独立可操作实验流程
answers.md 与 answers.pdf	E1至E16、G1至G10、L1至L8详解
data/protocol.json	看结果前固定的机制与协议
data/panel.csv	第0份面板的2880条记录
data/draws.npz	81份面板的全部原始随机抽样
data/splits.npz	每个协议的训练与测试行索引
data/split_membership.csv	逐行归属，含未使用行
data/hand.csv	四行训练加两行测试的纸笔表
data_integrity.json	固定数据的SHA256
experiment.py 与 experiment.ipynb	脚本与真实执行交互实验
audit.py 与 test-result.json	独立科学与程序自查
plots.py 与 figures	十幅实际图的重建代码和图
build_all.sh 与 配套构建文件	完整离线重建入口

experiment-result.json保留第0份逐行预测、系数、全部960次重复结果、Bootstrap抽样、两种权重和负面结果计数。数据文件不需要联网下载；它们都是原创合成数据。数据字典详见data/README.md。

3 准备独立环境

推荐用Python3.12的隔离环境。进入解压后的050目录：

```
python -m venv .venv
# Linux 或 macOS
source .venv/bin/activate
# Windows PowerShell 使用 .venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python experiment.py --out outputs/result.json
python audit.py --report outputs/result.json --out outputs/audit.json
```

偏好Anaconda或Miniconda时，可使用：

```
conda env create -f environment.yml
conda activate ml050-splits
python experiment.py --out outputs/result.json
```

作者已验证Python3.12.14、NumPy2.3.5、SciPy1.17.0、scikit-learn1.8.0、Matplotlib3.10.8、nbformat5.11.1、ipykernel7.4.0。environment.yml包含可选JupyterLab界面；作者测试的是新进程真实进程内内核，未测试JupyterLab浏览器、独立网络内核传输或完整Conda求解过程。不要把一份环境文件写出来就说所有平台安装已验证。

数值实验不需要Node、MathJax或WeasyPrint。只有重建PDF时才需要build-requirements.txt及package.json中的依赖，以及Pango和中文字体。安装失败时可以先阅读现有PDF并运行数值部分。

4 L1 先完成评价协议卡

在看分数之前，独立写四张小卡，每张含：部署时刻、已知设备、测试设备、训练日期、测试日期、标签到达、模型是否更新、主损失、权重。对照protocol.json，但不要把同一个“预测未来”复制四次。

主机制：设备g在t日先得到ID、日期和x，模型当日预测，标签到t+2才完成复核。时间协议在32日结束冻结模型，随后每日读当日x，预测33至47日；期间不重训。它不是在第32天提前拿到未来x的多步预测。

第0份面板只作详细演示；其余80份重新抽设备效应、输入和标签噪声。数据种子50017、划分种子50053、Bootstrap种子50059固定。不得看分数后换seed、改模型或删高误差设备来“还原”课本趋势。新问题应另建目录、另写协议，并保留原结果。

L1验收：能解释随机记录协议在观察期结束后作回顾性插值；整组协议是同一时期的新设备；时间协议是已知设备向前服务；联合协议是无设备历史的新设备未来冷启动。

5 L2 四行完整计算链

先打开hand.csv，只拿H1至H4训练。 θ 列顺序是b、w、vA、vB；H6的C是未知设备，必须编码为(1,2,0,0)。不要为它创建一个由测试y拟合的新偏置。

初始化0，计算每行预测、残差、半平方损失、局部导数。四行局部导数相加再除4，得到(-2.5,-1.75,-1,-1.5)。由于初始参数零，正则梯度也零。 $\eta=0.2$ 同步更新成(0.5,0.35,0.2,0.3)。

下一次预测必须用完整新 θ 逐行重算，得到0.7、1.05、0.8、1.15。半平方损失均值为1.681875；惩罚0.0315625，总目标1.7134375。最后只前向评价H5、H6，得到1.4、1.2。

```
import experiment as e
import audit
hand = e.hand()
for row in hand['rows']:
    print(row)
print(hand['gradient_mean'], hand['theta_next'])
print(hand['initial_objective'], hand['next_objective'])
print(audit.exact_hand())
```

Fraction精确参照给下一步目标5483/3200，最优岭系数(2,1,-1/3,1/3)。主实验使用增广最小二乘求最终解，手算的一步GD只是检查同一模型目标的局部计算，没有用 η 训练主实验的960个模型。

L2验收：提交四行表、聚合过程、正则项、两个测试前向。只写最终 θ 不通过；只写一个通用梯度公式但没有这四行数值也不通过。

6 L3 核对数据与划分 不先看MSE

```
import numpy as np
import experiment as e
c, draws, splits = e.load_data()
panel = e.panel(c, draws, 0)
for kind, (train, test) in splits.items():
    e.validate_split(panel, train, test, kind, c['fit_cutoff_day'])
    overlap = np.intersect1d(panel['group'][train], panel['group'][test])
    print(kind, len(train), len(test), len(overlap),
          panel['available'][train].max(), panel['day'][test].min())
```

正确训练、测试、设备交集依次为：random 2160、720、60；group 2160、720、0；time 1860、900、60；group_time 1395、225、0。时间和联合的最大标签到达日32、首个预测日33。随机和整组的最大标签到达日49，不符合第32日上线可用性，但符合声明的回顾性目标。

time未用120行，正好是所有设备第31、32日；group_time未用1260行，包含45个训练设备的31至47日765行，以及15个留出设备的0至32日495行。可以保存在原始表里而不交给训练，不能把这些未用行称为删除坏例。

用CSV软件只读查看split_membership.csv也能检查同样的归属。若应用软件自动重写CSV的小数或换行，load_data的字节校验会失败。不要直接修改哈希消除警报；保留原数据再做另一个明确命名的实验。

L3验收：说明为什么只有索引不相交还不够，以及同设备交集60何时合法、何时表示目标错配。另算标签延迟5天时32日截止最晚可训练27日。

7 L4 主实验和负面结果

```
python experiment.py --out outputs/ordinary.json
python -O experiment.py --out outputs/optimized.json
python audit.py --report outputs/ordinary.json --out outputs/audit.json
python -O audit.py --report outputs/optimized.json --out outputs/audit-0.json
```

-O会移除普通Python assert，本讲输入、划分和检查失败均通过显式异常实现，因此优化模式仍执行同样检查。输出采用严格JSON与原子替换；不允许覆盖随包源码、数据或PDF。指定新输出路径时先验证路径，序列化失败保留已有文件。

普通与优化模式的结果应一致。也可在其他目录调用本机050/experiment.py的绝对路径，默认数据按脚本位置寻找；相对--out仍相对于调用者当前目录。请不要复制作者机器上的绝对路径，改成自己的解压位置。

读取三个模型在四个协议的成绩：

```
import json
from pathlib import Path
r = json.loads(Path('outputs/ordinary.json').read_text())
for kind, block in r['main'].items():
    print(kind, {name: value['mse']
                 for name, value in block['models'].items()})
print(r['negative_results'])
```

第0份Personalized依次约0.636795、1.847585、1.496320、2.938758。这里按random、group、time、group_time顺序，不依赖JSON的字母排序。对应80份均值约0.636406、2.140160、1.514119、3.193832。

80份中Personalized在整组有34次、联合有38次差于Pooled。Trend在联合也有3次差于Personalized。所有960次都在replications中，而非只留下成功改善的一部分。图5会展示全部点；如果你的方向与书中不同，先查数据和协议，不要改seed碰运气。

L4验收：写出能与不能进行的比较。同协议内比较三个固定模型共享评价数据；跨协议同时改变目标与信息，不能把差值全部解释为算法改善或某一种泄漏的大小。

8 L5 独立库与科学参照

主代码用自己构造的指示列及NumPy lstsq。审计中的独立路线从原始group另用OneHotEncoder, 仅fit训练组, 设置handle_unknown='ignore', 再用真实Ridge(alpha=1, fit_intercept=True, solver='svd')拟合。第三条路线用SciPy的gelsy带列主元QR求增广系统。

```
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import Ridge
train, test = splits['group']
enc = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
g_train = enc.fit_transform(panel['group'][train, None])
g_test = enc.transform(panel['group'][test, None])
X = np.column_stack([panel['x'][train], g_train])
Xt = np.column_stack([panel['x'][test], g_test])
model = Ridge(alpha=1.0, fit_intercept=True, solver='svd')
model.fit(X, panel['y'][train])
pred = model.predict(Xt)
print(np.mean((pred-panel['y'][test])**2))
print(g_test.sum())
```

输出MSE约1.847585, g_test.sum()为0, 因为整组测试设备训练未见。未知组回退是预先定义的一部分, 不是测试后补救。接口支持无惩罚alpha=0, 本教学fit主动要求alpha>0以保证当前完整指示编码下的稳定唯一解; 不要把教学包装的边界误说成岭理论只支持正数。

audit.py还调用真正的GroupShuffleSplit检查无设备交叉, 调用TimeSeriesSplit在48个日索引上验证gap语义。库的随机划分不必与本包固定索引一样, 因为随机算法和抽样顺序不同; 必须相同的是可核查的组边界, 不是恰巧相同的名单。

L5验收: 报12份主例的sklearn及SciPy最大预测误差, 并说明独立编码为何比把待测函数调用两次更可靠。作者结果最大误差约 6.75×10^{-14} ; 四行下一步梯度的中心差分最大差约 1.47×10^{-10} , 使用合理容差而不是要求所有末位完全相同。

9 L6 评价权重和Bootstrap

先做两设备精确例: U有8行、V有2行, A的组损失1和9, B都为4。分别得到行MSE A=2.6、B=4, 组MSE A=5、B=4。写出抽一行与抽一台设备的不同概率。

然后读取主例Bootstrap:

```
b = r['bootstrap']
for key in ['rows', 'independent_groups', 'row_sd', 'group_sd',
            'row_percentile_95', 'group_percentile_95', 'estimand']:
    print(key, b[key])
```

本实验固定整组拟合Personalized，比较720行有放回抽样与15个整设备有放回抽样，各800次，均不重训。行标准差约0.09779，组标准差约0.42365。后者更宽符合设备内依赖的机制，但没有在这里验证百分位区间的95%覆盖率，尤其15组并不多。

组级重采样模拟测试设备构成变化；80份面板则重新抽训练和测试，再训练所有模型。它们不是两条互换的“误差棒”。若现实设备共享一次电网冲击或共同维护政策，设备之间也未必独立，仅按设备重采样可能仍不够。

L6验收：写明固定了什么、重抽了什么、是否重训、权重是否改变、覆盖是否已检验。不能只说“使用Bootstrap所以可靠”。

10 L7 窗口与时间接口

窗口例使用历史 $[t-3, t]$ ，标签是 $t+1, t+2$ 两天汇总，与主面板“当日状态标签延迟两天到达”区别开。在32日结束训练时，只按可用性需 $t \leq 30$ ；若与首个测试 $t=33$ 的完整底层支持 $[30, 35]$ 严格无交集，则需 $t+2 < 30$ ，即 $t \leq 27$ 。

实际库检查：

```
from sklearn.model_selection import TimeSeriesSplit
for train_days, test_days in TimeSeriesSplit(
    n_splits=3, test_size=8, gap=2).split(np.arange(48)):
    print(train_days[-1], test_days[0], test_days[-1])
    train_rows = np.flatnonzero(np.isin(panel['day'], train_days))
    test_rows = np.flatnonzero(np.isin(panel['day'], test_days))
    print('Rows:', len(train_rows), len(test_rows))
```

日边界三次依次为(21,24,31)、(29,32,39)、(37,40,47)。这里 $\text{gap}=2$ 确实是两天，因为API输入一行就是一天；映射回面板时才展开成每台设备记录。本段演示API语义，未将这些新划分混入主实验四份结果，也未进行交叉验证选参。

L7验收：解释为什么直接在按设备排列的2880行上运行TimeSeriesSplit很危险；解释共享过去已知 x 可以是合法向前预测，而共享同一待评价答案可能形成泄漏。不能把所有重叠都一律说成非法，也不能忽略其对统计独立性的影响。

11 图像和Notebook真实运行

```
python plots.py --report outputs/ordinary.json --directory outputs/figures
python execute_notebook.py --out outputs/fresh-kernel.ipynb
```

十张图顺序为划分地图、标签可用时间、四行更新、主例MSE、80份重复、偏移与残差、设备分组错误、Bootstrap单位、评价权重、窗口支持。注意讲义为了推导顺序先讨论图9再图8；文件编号与来源数据不变。

交付experiment.ipynb保留执行输出。新运行需要从头执行全部代码单元，不依赖旧内核变量。首个代码单元先以标准库校验代码与数据SHA256，随后才导入数值依赖。每项图像由程序重建并显示，不是仅保留一张没有代码的截图。

execute_notebook.py在新的Python进程启动真实ipykernel InProcessKernel，执行并保存输出到指定文件，失败时返回非零退出码和具体代码格。它不测试浏览器Jupyter或外进程网络传输。若使用JupyterLab，请在050目录打开，选择重启内核并运行全部；其他工作目录会由首格明确报缺失文件，不会偷偷使用别处同名数据。

若需修改Notebook文本后重建源，使用build_notebook.py；它会清空原输出，所以日常学习只执行现有Notebook即可。原始交付包先保留，避免把你的探索覆盖成官方参考。

12 常见输入与失败保护

本教学fit支持非空一维有限实数x、日期和y，非负整数设备ID，各列等长，每向量不超过200000项，alpha为有限正标量。它不是通用数据清洗库，不接受自动把字符串解析成数值，不对NaN偷偷填0。模型名只接受pooled、personalized、trend。

审计验证单行、常数x、未知设备、训练行逆序等常见合理情况；也验证空输入、NaN、长度不齐、非整数设备ID、非正alpha、行交叉、设备交叉、忽略标签延迟及未来训练过去评价会明确失败。主模型的设备类别由训练集获得，测试标签被大幅改动也不能影响训练系数。

输出保护拒绝覆写随包课程文件、符号链接和多硬链接；JSON不允许NaN/Infinity，序列化错误时原文件保留。这是防误操作的课程边界，不是恶意并发环境的安全保证。generate_data.py要求新建空目录，默认不会改写data。

如果数据hash不符，先比较原包；如果group leakage，先问协议是否真要测新设备；如果unavailable training label，核对label_available_day；如果图形出乎预期，检查目标、符号、权重与阴影含义。不要把每个低分都当成优化器没收敛。

13 可选完整重建

准备好数值、Notebook和PDF依赖后，可运行：

```
python -m pip install -r build-requirements.txt
npm install
bash build_all.sh
```

build_all.sh将数值、普通与优化模式、审计、十图、源Notebook、新内核、数据再生成与PDF重建串起。主要结果写到outputs/rebuild，原始数据与交付图不会静默覆盖。PDF构建从本讲自己的Markdown和figures读取；验证图像重建相同后才对图文一致作结论。

Node.js和MathJax3.2.2负责把公式转为本地SVG；WeasyPrint70排版可信Markdown。需要Pango、Noto Sans/Serif CJK中文字体和DejaVu Sans Mono。构建不调用网络公式服务，不需要XeLaTeX。跨平台字体与排版可能略有不同，必须重新逐页检查，生成成功并不等于没有裁切。

generate_data.py可在新目录重建数据；它将ZIP内部日期固定，因此当前环境内npz字节也可复现。改变NumPy版本时首先比较数组数值及协议，不把压缩字节不同自动解释为统计结论改变。固定种子是重现工具，不是稳健性证明。

14 L8 写研究报告并完成验收

写300至500字报告，至少包括：评价目标、标签时序、四个协议、一个同协议模型比较、一个保留的负面结果、权重与独立单位、科学参照以及现实外推限制。不要把四种成绩只写成“最好是随机切分”。

完成检查：

- 四行梯度链和两次前向、正则项、未知设备编码全部可重算
- 2880行每个协议的归属清楚，时间截止看标签到达而非只看事件日期
- 第0份12个MSE及另外80份960个拟合完整保留
- 独立Fraction、SciPy与sklearn对照通过，测试标签变更不影响拟合
- 行权重与组权重分清，Bootstrap条件与覆盖局限明确
- 普通、-O、不同cwd和新Notebook内核真正执行，错误未静默跳过
- 数据、图、PDF能从完整包恢复，十幅图及最终PDF逐页检查
- 结果没有声称模拟器等同现实，也未把已见测试分数当无选择偏差的最终部署证据

答案册提供所有E题、G题和L题的完整解释。研究型L8可有不同合理方案，关键是信息可获得、目标明确、协议可检查，且没有用未知未来来替过去作决定。